```
<form accept-charset="UTF-8" action="/search" method="get">
  <input name="utf8" type="hidden" value="&#x2713;" />
  <label for="q">Search for:</label>
  <input id="q" name="q" type="text" />
  <input name="commit" type="submit" value="Search" />
</form>
```

For every form input, an ID attribute is generated from its name ("q" in above example). These IDs can be very useful for CSS styling or manipulation of form controls with JavaScript.

Besides `text_field_tag` and `submit_tag`, there is a similar helper for *every* form control in HTML.
Always use "GET" as the method for search forms. This allows users to bookmark a specific search and get back to it. More generally Rails encourages you to use the right HTTP verb for an action.

1.2 Multiple Hashes in Form Helper Calls

The `form_tag` helper accepts 2 arguments: the path for the action and an options hash. This hash specifies the method of form submission and HTML options such as the form element's class.
As with the `link_to` helper, the path argument doesn't have to be a string; it can be a hash of URL parameters recognizable by Rails' routing mechanism, which will turn the hash into a valid URL. However, since both arguments to `form_tag` are hashes, you can easily run into a problem if you would like to specify both. For instance, let's say you write this:

```
form_tag(controller: "people", action: "search", method: "get", class:
"nifty_form")
# => '<form accept-charset="UTF-8"
action="/people/search?method=get&class=nifty_form" method="post">'
```

Here, `method` and `class` are appended to the query string of the generated URL because even though you mean to write two hashes, you really only specified one. So you need to tell Ruby which is which by delimiting the first hash (or both) with curly brackets. This will generate the HTML you expect:

```
form_tag({controller: "people", action: "search"}, method: "get",
class: "nifty_form")
# => '<form accept-charset="UTF-8" action="/people/search"
method="get" class="nifty_form">'
```

1.3 Helpers for Generating Form Elements

Rails provides a series of helpers for generating form elements such as checkboxes, text fields, and radio buttons. These basic helpers, with names ending in `_tag` (such as `text_field_tag` and `check_box_tag`), generate just a single `<input>` element. The first parameter to these is always the name of the input.
When the form is submitted, the name will be passed along with the form data, and will make its way to the `params` hash in the controller with the value entered by the user for that field. For example, if the form contains `<%= text_field_tag(:query) %>`, then you would be able to get the value of this field in the controller with `params[:query]`.
When naming inputs, Rails uses certain conventions that make it possible to submit parameters with non-scalar values such as arrays or hashes, which will also be accessible in `params`. You can read more about them in chapter 7 of this guide. For details on the precise usage of these helpers, please refer to the API documentation.

1.3.1 Checkboxes

Checkboxes are form controls that give the user a set of options they can enable or disable:

```
<%= check_box_tag(:pet_dog) %>
<%= label_tag(:pet_dog, "I own a dog") %>
<%= check_box_tag(:pet_cat) %>
<%= label_tag(:pet_cat, "I own a cat") %>
```